

Course: Data Structures (ETCCDS202)

Unit: 4 – Non-Linear & Advanced Data Structures

Student Name: Jai Kashyap

Roll No: 2501630019

Section: A

1. Introduction

This assignment implements a **Data Management Mini Toolkit (DMMT)** using Python. It includes three major components:

- Binary Search Tree (BST)
- Graph (Adjacency List with BFS & DFS)
- Hash Table (with Separate Chaining)

These data structures are widely used in real-world systems like databases, search engines, caching, and networking.

2. Binary Search Tree (BST)

2.1 Operations Implemented

- Insert
- Search
- Delete (all 3 cases)
- Inorder Traversal

2.2 Why Inorder Traversal Gives Sorted Output

In a BST:

- Left subtree contains smaller values
- Right subtree contains larger values

Inorder traversal follows:

Left → Root → Right

This naturally prints elements in **ascending (sorted) order**.

2.3 Deletion Cases

1. **Leaf Node (No Child)**
→ Node is directly removed
 2. **One Child**
→ Node is replaced by its child
 3. **Two Children**
→ Replace node with **inorder successor** (smallest in right subtree)
-

2.4 Time Complexity

- Average case: $O(\log n)$
 - Worst case (skewed tree): $O(n)$
-

3. Graph Implementation

3.1 Representation

The graph is implemented using an **Adjacency List**, where each node stores:

- Neighbor nodes
 - Edge weights
-

3.2 BFS (Breadth-First Search)

- Uses a **queue**
- Traverses level by level

Real-World Use Case

- Finding **shortest path in unweighted graphs** (e.g., GPS navigation)
-

3.3 DFS (Depth-First Search)

- Uses **recursion or stack**
- Goes deep before backtracking

Real-World Use Case

- **Cycle detection** (e.g., detecting loops in networks or dependencies)

3.4 Difference Between BFS and DFS

Feature	BFS	DFS
Structure	Queue	Stack/Recursion
Traversal	Level-wise	Depth-wise
Use Case	Shortest path	Cycle detection

4. Hash Table (Separate Chaining)

4.1 Concept

A hash table stores data as **key-value pairs** using a hash function.

Hash function used:

$\text{index} = \text{key} \% \text{table_size}$

4.2 Collision

A **collision** occurs when:

- Two keys map to the same index

Example:

$10 \% 5 = 0$

$15 \% 5 = 0$

Both go to index 0 → collision

4.3 Separate Chaining

- Each index stores a **list (chain)**
- Colliding elements are stored in that list

Why It Helps

- Prevents data loss
 - Allows multiple elements at same index
-

4.4 Time Complexity

- Average case: **$O(1)$**
 - Worst case: **$O(n)$** (if many collisions occur)
-

5. Observations

- BST provides **efficient sorted storage**
- Graph traversal helps in **network exploration**
- Hash table provides **fast lookup**

Together, these structures simulate real-world systems like:

- Databases (BST, B-Trees)
 - Social networks (Graphs)
 - Caching systems (Hash Tables)
-

6. Conclusion

This assignment helped in understanding how advanced data structures work together in real applications. It improved knowledge of:

- Searching and sorting
- Traversing networks
- Efficient data storage and retrieval